



CORE JAVA PROGRAMMING

# Exception Handling in Java

| Strategies for Robust & Reliable Code

## INSTRUCTOR

 Shubham Rawat

## DURATION

 100 Minutes

## MODULE FOCUS

 Error & Exception Handling

## IN THIS SESSION:

 Hierarchy

 try-catch-finally

 Checked vs Unchecked

 Best Practices

```
try {  
    execute();  
} catch (Exception e) {  
    handle(e);  
} finally {  
    cleanup();  
}
```





# What is an Exception?

Core Java: Exception Handling

“ An **exception** is an event that occurs during the execution of a program that **disrupts the normal flow** of instructions.

## REAL WORLD ANALOGIES



### ATM Transaction

User requests cash > Machine has **Insufficient Funds** > Process stops.



### Online Download

Downloading file > **Internet Disconnects** > Download fails.

```
Main.java

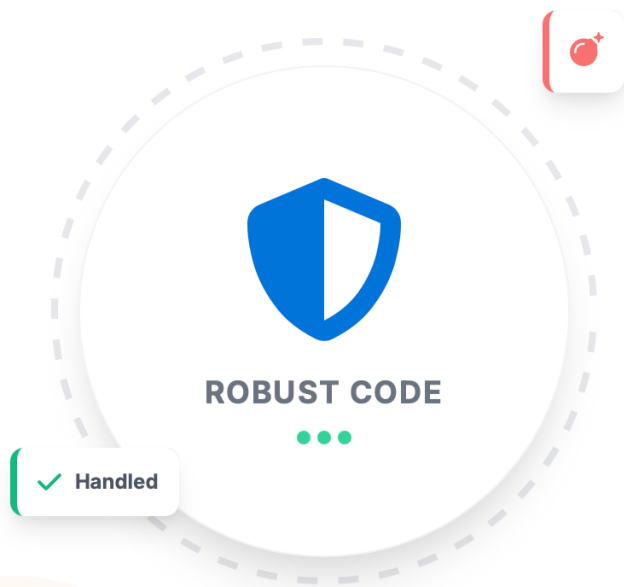
public class Main {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;
        // ⚠ This line causes an Exception
        int result = a / b;
        // The lines below will NOT execute
        System.out.println("Result: " + result);
    }
}
```

```
Exception in thread "main"
java.lang.ArithmeticException: / by zero
    at Main.main(Main.java:6)
```

⚠ Program Crashes Here

# Why Exception Handling?

Core Java: Concepts



*"Without handling, the entire system is vulnerable to a single error."*



## Prevents Abnormal Termination

Prevents the program from crashing abruptly, keeping the application responsive even when errors occur.



## Isolates Failures

Separates error-handling code from regular business logic, ensuring one failure doesn't stop the whole system.



## Better User Experience

Allows displaying friendly, meaningful error messages to users instead of cryptic stack traces.



## Ensures Resource Cleanup

Uses `finally` blocks to safely close files, database connections, and network sockets.



## Improves Reliability

Enhances the overall stability and quality of the software, making it easier to maintain and debug.

## Exception RECOVERABLE PROBLEM

### ✓ Can be Handled

Program can catch it and continue execution normally.

### </> Application Level

Caused by issues in program code or invalid user input.

### Types

Checked (Compile-time) & Unchecked (Runtime).

#### EXAMPLES

ArithmeticException

IOException

SQLException

VS

## Error CRITICAL FAILURE

### ✗ Irrecoverable

Cannot usually be handled; the program typically crashes.

### System Level

Caused by lack of system resources or JVM failures.

### ⊘ Handling

Should NOT try to catch Errors in normal application logic.

#### EXAMPLES

OutOfMemoryError

StackOverflowError

```
// Recoverable
try { readFile(); }
catch (IOException e) { ... }
```

```
// Fatal
// Do not catch this!
throw new OutOfMemoryError();
```

# Types of Exceptions

Core Java: Classification

## Built-in JAVA STANDARD LIBRARY

- ✓ **Provided by Java SDK**  
Ready to use classes available in java.lang package.
- ⚙️ **Common Errors**  
Handles generic programming issues like null pointers or math errors.
- 🛡️ **Standardized**  
Universally understood by all Java developers.

### EXAMPLES

NullPointerException

IOException

ArithmeticException

```
// Implicitly thrown by JVM
int result = 10 / 0;
// Throws ArithmeticException
```

VS

## User-defined CUSTOM BUSINESS LOGIC

- 🔨 **Created by Developers**  
Custom classes to handle specific application requirements.
- 👤 **Inheritance**  
Must extend 'Exception' (checked) or 'RuntimeException' (unchecked).
- 💡 **Domain Specific**  
Makes code more readable and meaningful for the business context.

### EXAMPLES

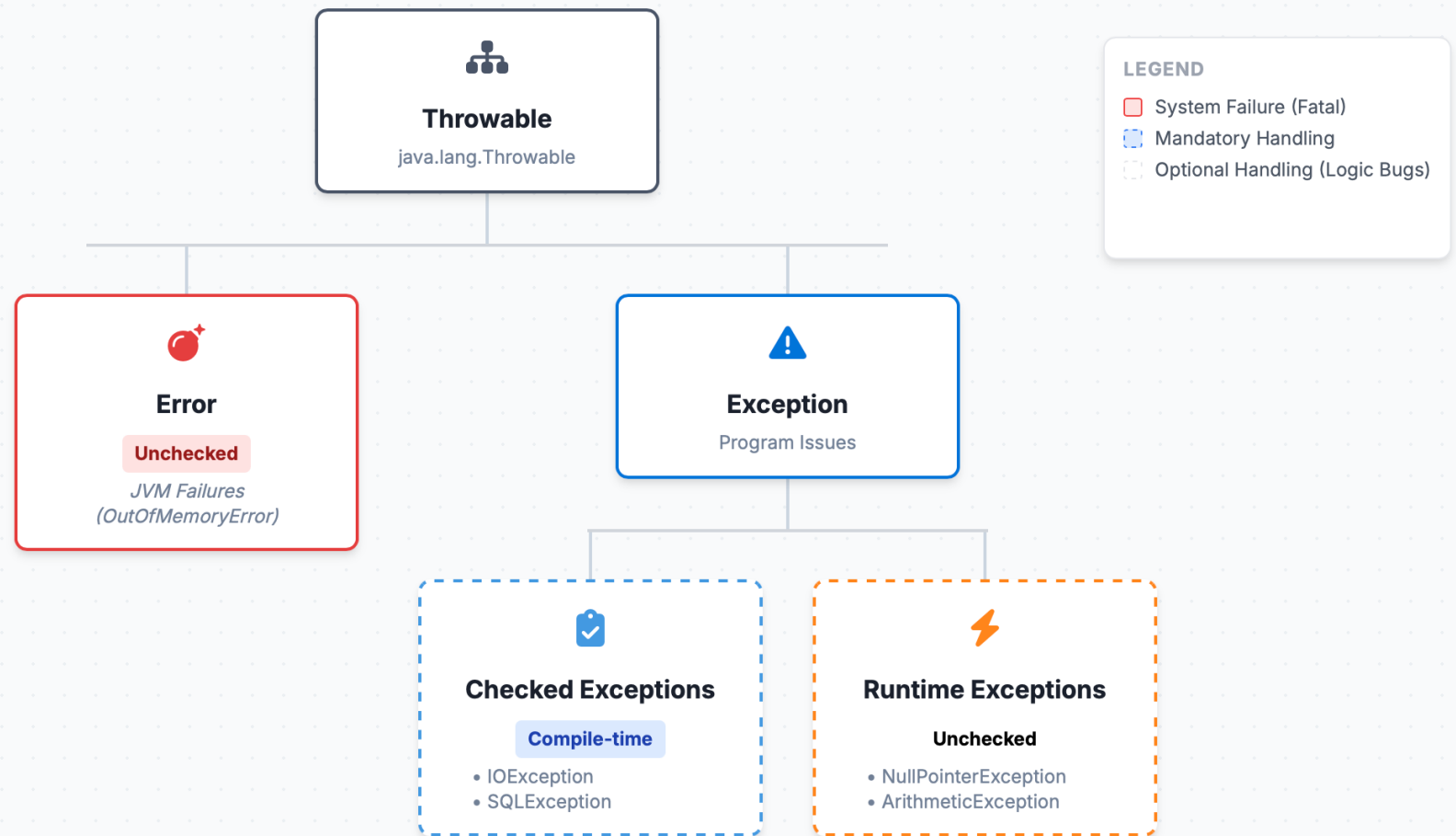
InvalidAgeException

InsufficientFunds

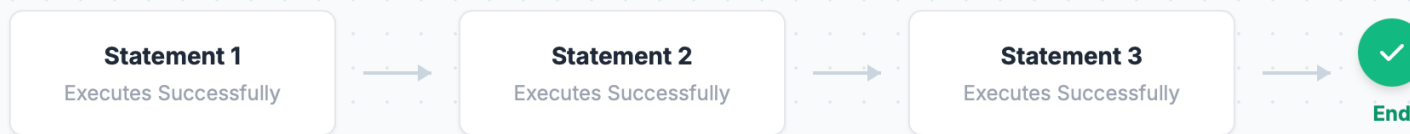
```
// Manually thrown by dev
throw new InvalidAgeException("Too young");
```

# Exception Hierarchy

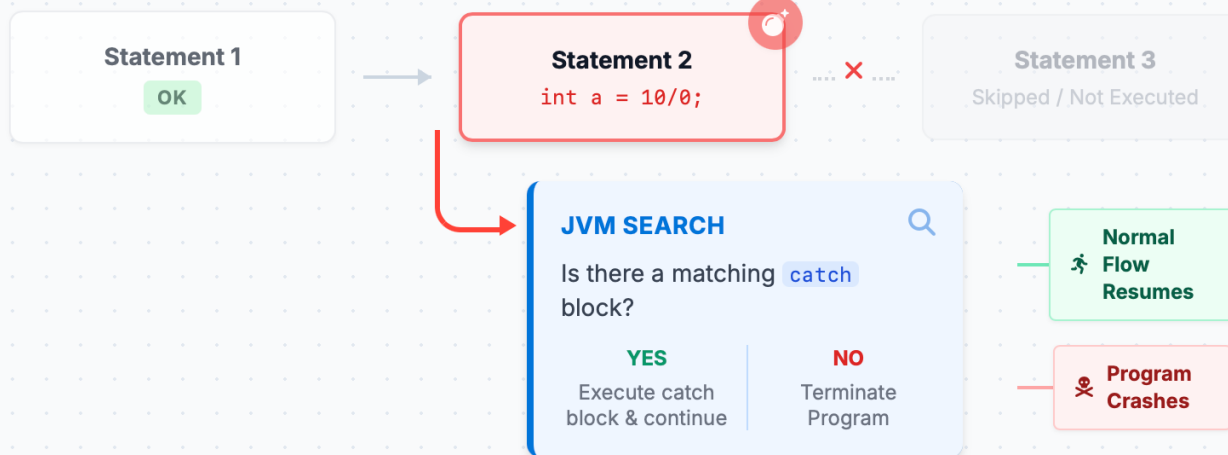
Core Java: Structure



## → Normal Execution Flow

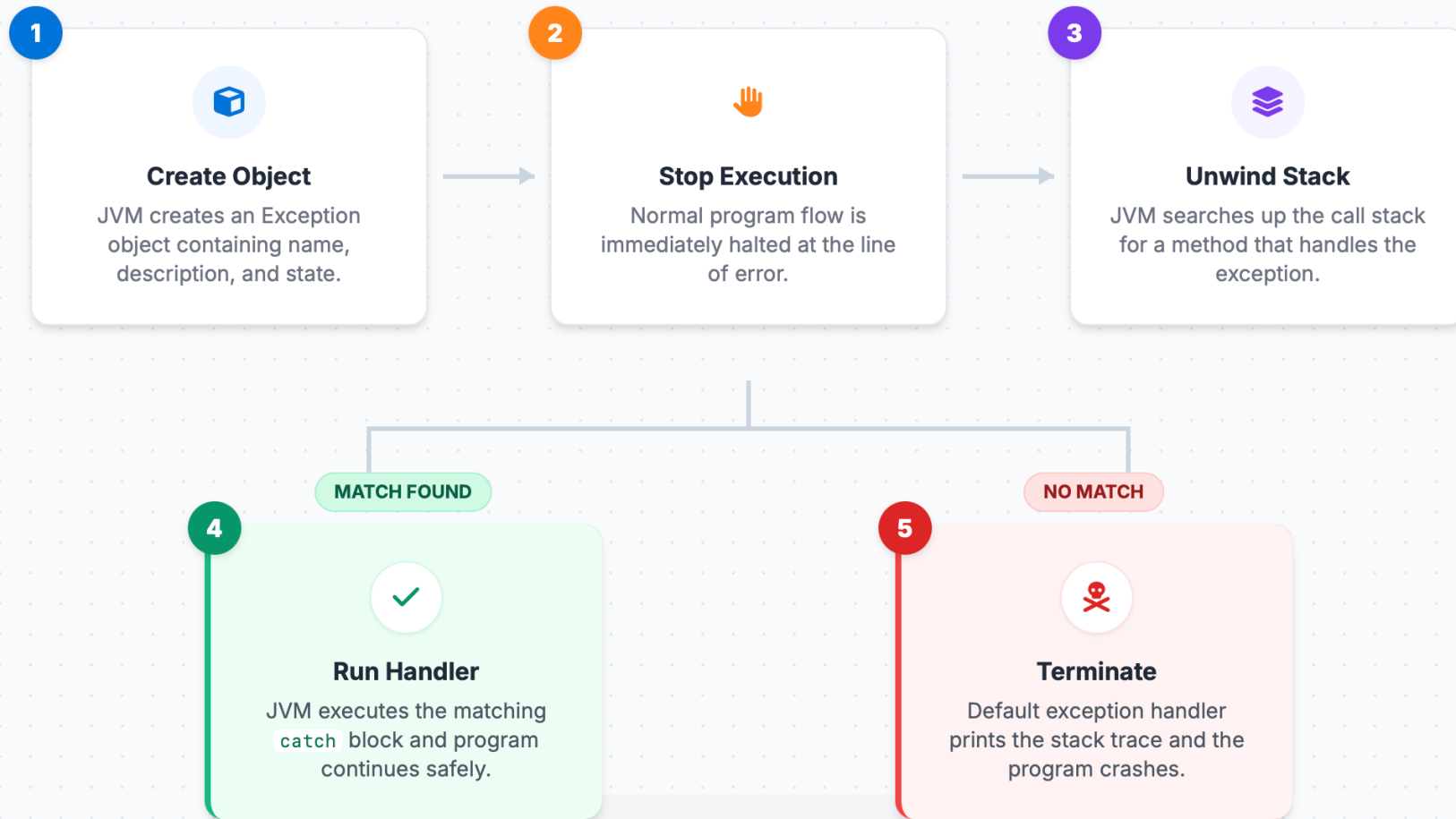


## ! Exception Execution Flow



# How the JVM Handles Exceptions

Core Java: Internal Mechanism



i The JVM runtime system is responsible for finding the appropriate exception handler.



# try-catch: Syntax & Example

Core Java: Exception Handling

## </> The Mechanism

**try** Encloses **risky code** that might throw an exception.

**catch** Defines a block of code to **handle** the exception if it occurs.

## BEST PRACTICES



### Be Specific

Catch specific exceptions (e.g., `ArithmeticException`) rather than generic `Exception`.



### Avoid Empty Catch

Never leave a catch block empty. At minimum, print the stack trace.

Calculator.java

```
public class Calculator {  
    public static void main(String[] args) {  
        try {  
            int a = 10;  
            int b = 0;  
            // ⚠️ Risky code below  
            int result = a / b;  
            System.out.println("Result: " + result);  
        }  
        catch (ArithmeticException e) {  
            // ✅ Handling code  
            System.out.println("Error: Cannot divide by  
zero!");  
        }  
        System.out.println("Program continues  
safely...");  
    }  
}
```

Error: Cannot divide by zero!

Program continues safely...

✅ Execution Resumed



# Multiple **catch** Blocks

Core Java: Exception Handling

## The Golden Rule

When using multiple catch blocks, you must order exceptions from **most specific** to **most general**.

 Parent exceptions (like `Exception`) catch everything, so if placed first, child exceptions become unreachable.

## HIERARCHY FLOW

`ArithmeticException`

Specific (Child)



`RuntimeException`

Parent



`Exception`

General (Grandparent)

## ★ Bonus: Multi-catch Block

Since Java 7, catch multiple types in one block using pipe `|`.

```
catch (ArithmeticException | NullPointerException e)
{ ... }
```

MultipleCatchDemo.java

```
public class MultipleCatchDemo {
    public static void main(String[] args) {
        try {
            int[] arr = new int[3];

            // ⚠️ Creates ArrayIndexOutOfBoundsException
            arr[5] = 10;

            int val = 10 / 0; // Skipped
        }
        // ✅ Specific handler checks first
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Bad index accessed");
        }
        catch (ArithmeticException e) {
```

>\_ CONSOLE OUTPUT

```
Error: Bad index accessed
Program continues...
```

## What is it?

A block that follows try or catch and **always executes**, regardless of whether an exception occurred or not.

## WHY USE IT? (CLEANUP)



### Closing Files

Ensures file streams are closed to prevent leaks.



### Database Connections

Terminates active connections to the database.



### Socket Connections

Releases network ports and resources.

 **Note:** finally runs even if there is a return statement in the try block!

```
ResourceCleanup.java

public void readFile() {
    FileInputStream fis = null;
    try {
        fis = new FileInputStream("data.txt");
        // Read file operations...
    }
    catch (IOException e) {
        System.out.println("Error: " + e.getMessage());
    }
    finally {
        // ⚠ Always runs for cleanup
        if (fis != null) {
            fis.close();
            System.out.println("Resource closed.");
        }
        System.out.println("Finally executed");
    }
}
```

```
Error: data.txt (The system cannot find the file specified)
> Finally executed
(Notice: Finally ran even after the exception)
```



# The **throw** Keyword

Core Java: Exception Handling



The **throw** keyword is used to explicitly throw an exception from a method or any block of code.

## WHY USE IT?



### Validation Logic

Enforce business rules (e.g., age must be 18+).



### Custom Errors

Throw specific or user-defined exceptions.

## SYNTAX

```
throw new ExceptionType("Error Message");
```

AgeValidator.java

```
public class AgeValidator {  
    // Method to check age eligibility  
    static void validate(int age) {  
        if (age < 18) {  
            // ⚠️ Manually throwing exception  
            throw new IllegalArgumentException("Not Eligible");  
        }  
        else {  
            System.out.println("Welcome to Vote!");  
        }  
    }  
    public static void main(String args[]) {  
        validate(13); // Passing invalid age  
    }  
}
```

```
Exception in thread "main"  
java.lang.IllegalArgumentException: Not Eligible  
    at AgeValidator.validate(AgeValidator.java:6)  
    at AgeValidator.main(AgeValidator.java:12)
```

**i** Program terminates immediately at line 6



# The **throws** Keyword

## **i** Declaration Responsibility

Used in a method signature to declare that the method **might cause** specific exceptions. It passes the responsibility of handling the exception to the **caller** of the method.

## throw vs. throws

| Feature     | throw                         | throws                        |
|-------------|-------------------------------|-------------------------------|
| Location    | Inside method body            | Method signature              |
| Purpose     | Explicitly throw an exception | Declare possible exceptions   |
| Quantity    | Single exception instance     | Multiple classes (comma sep.) |
| Propagation | Stops execution immediately   | Warns compiler / caller       |

```
FileHandler.java

import java.io.*;

public class FileHandler {

    // 1. Declare that this method is "risky"
    static void readFile(String fileName) throws IOException
    {
        File file = new File(fileName);
        FileInputStream fis = new FileInputStream(file);
        // If file missing, IOException occurs here
    }

    public static void main(String[] args) {
        try {
            // 2. Caller MUST handle the risk
            readFile("data.txt");
        } catch (IOException e) {
            System.out.println("File error: " + e);
        }
    }
}
```

**File error: java.io.FileNotFoundException: data.txt (The system cannot find the file specified)**

## Checked

COMPILE-TIME VERIFIED

-  **Compile-Time Check**  
Compiler forces you to handle these. Code won't compile otherwise.
-  **Mandatory Handling**  
Must be handled with `try-catch` or declared with `throws`.
-  **External Contingencies**  
Usually represent valid conditions outside program control (IO, DB).

### EXAMPLES

`IOException` `SQLException` `ClassNotFoundException`

```
// Must handle or declare
void readFile() throws IOException {
    ...
}
```

VS

## Unchecked

RUNTIME OCCURRENCE

-  **Runtime Only**  
Compiler ignores them. Occur only during program execution.
-  **Optional Handling**  
Handling is not enforced. Usually indicates logic errors.
-  **Inheritance**  
Extends `RuntimeException` (and its subclasses).

### EXAMPLES

`NullPointerException` `ArithmeticException` `IndexOutOfBoundsException`

```
// Logic error (Bug)
String s = null;
int len = s.length(); // NPE
```



Custom exceptions designed to handle specific **business logic** or domain rules.

Useful when built-in Java exceptions (like `ArithmeticException`) don't accurately describe the error condition.

## HOW TO IMPLEMENT



### 1. Extend Exception Class

Extend `Exception` (Checked) or `RuntimeException`.



### 2. Create Constructor

Pass the error message string to the parent class using `super(msg)`.



### 3. Throw Manually

Use the `throw` keyword to trigger it in your code.

VotingSystem.java

```
// 1. Create the Custom Exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String str) {
        super(str); // Pass message to parent
    }
}

// 2. Usage in Application Logic
public class Main {
    static void validate(int age) throws
        InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Not eligible");
        }
        System.out.println("Welcome to vote!");
    }
    public static void main(String args[]) {
        try { validate(15); }
        catch(Exception e) { System.out.println(e); }
    }
}
```

**InvalidAgeException: Not eligible**



## Safe Parsing with Cleanup

A robust pattern for handling uncertain inputs while ensuring program integrity.

### 🛡️ The Goal

Convert a String to an Integer (`Integer.parseInt`) without crashing if the input is invalid (e.g., "abc").

### 🧹 The Cleanup

Use `finally` to ensure critical code executes regardless of success or failure (e.g., closing connections, logging completion).

### EXECUTION FLOW

- **Attempt Parsing**  
Inside try block
- **Handle Failure**  
Catches `NumberFormatException` if input is bad
- **Always Finalize**  
`finally` runs 100% of the time

SafeParser.java

```
public class SafeParser {  
    public static void main(String[] args) {  
        String input = "abc"; // ⚠️ Invalid number  
  
        try {  
            System.out.println("Parsing: " + input);  
            int num = Integer.parseInt(input);  
            System.out.println("Success: " + num);  
        }  
        catch (NumberFormatException e) {  
            System.out.println("❌ Error: Invalid format");  
        }  
        finally {  
            // This executes NO MATTER WHAT  
            System.out.println("✅ Cleanup: Done");  
        }  
    }  
}
```

### CONSOLE OUTPUT

```
Parsing: abc  
❌ Error: Invalid format  
✅ Cleanup: Done
```



# Common Mistakes & Best Practices

Core Java: Exception Handling



## Common Pitfalls

AVOID THESE PATTERNS

- ❌ **Swallowing Exceptions**  
Leaving the catch block empty hides bugs.
- ❌ **Catching 'Exception'**  
Too generic; catches RuntimeExceptions too.
- ❌ **Wrong Order**  
Catching Parent (Exception) before Child.
- ❌ **Confusing Keywords**  
Using throw in signature instead of throws.

```
// ❌ BAD PRACTICE
try { ... }
catch (Exception e) {
    // Empty block...
}
```



## The Fix

CORRECT USAGE

- ✅ **Handle or Log**  
Always print stack trace or log the error.
- ✅ **Specific First**  
Catch IOException before Exception.
- ✅ **Cleanup Resources**  
Always use finally to close DB/Files.
- ✅ **Know Your Keywords**  
throw new (Object) vs throws (Decl).

```
// ✅ GOOD PRACTICE
catch (IOException e) {
    e.printStackTrace();
}
finally { conn.close(); }
```



# Exam Preparation & Viva Questions

Core Java: Revision



## Common Interview Questions

### Q1. What is the difference between Error and Exception? HIGH FREQ

Focus on: Recovery (possible vs impossible) and Origin (System vs Program).

### Q2. Explain Checked vs Unchecked Exceptions.

Focus on: Compile-time check vs Runtime check. Mention `IOException` vs `NullPointerException`.

### Q3. What is the purpose of the 'finally' block? MUST KNOW

Focus on: Cleanup code (closing connections) that runs regardless of exception occurrence.

### Q4. Difference between 'throw' and 'throws'?

Focus on: Action (throwing an object) vs Declaration (method signature warning).



## One-Line Definitions

### Exception

An abnormal event that disrupts the normal flow of program execution.

### Checked Exception

Exceptions verified by the compiler at compile-time (e.g., `IOException`).

### Unchecked Exception

Exceptions that occur at runtime and are not checked by the compiler (e.g., `ArithmeticException`).

### try-catch

Blocks used to enclose risky code and handle specific exceptions gracefully.

### Stack Trace

A report of the active stack frames at a certain point in time during execution.